– Independent Work Report 2026 –

AUTO-I3RAB: Using a Joint Hidden Markov Model to Predict Ending Diacritical Marks in Arabic Text

Ayaat Al-Yasseri
Adviser: Christiane Fellbaum
NES Advisor: Mounia Mnouer

Abstract

As the field of NLP rapidly grows, the need for efficient models in areas involving less-researched languages such as Arabic have increased. However, given Arabic's complex grammatical structure, many models are unable to properly handle all forms of Arabic, including Modern Standard Arabic (MSA), the form designated for use in writing, politics, business, and other forms of official business. These models frequently miss key components of the grammar structure (referred to as I'rab) that lie in the diacritical marks, which are almost always omitted and instead assumed to be implicitly known. This hinders meaning and understanding of the text, in both the human language component and NLP component. This project proposes to create a two-layer Hidden Markov Model that hinges on learning the relationship between the word, part of speech (POS), and ending diacritical mark, with a joint Viterbi layer that takes into account the probabilities learned in the two layers. The underlying assumption of the joint model is that added grammatical context in the form of POS-tagging is able to boost prediction accuracy of the ending diacritical mark. We find that using a weighted function to calculate emission probability of the joint layer, at around $\alpha = 0.6$*, the overall accuracy does increase by around* $2-4\%$*, with an increase in unknown accuracy of around* $10-12\%$*, indicating that the additional context does increase accuracy of the model in comparision to a regular (word, mark) Hidden Markov Model.*

1. Introduction

As one of the world's most spoken languages[19], NLP research of Arabic is much sparser than its counterparts, leading to some arguing it should be classified as a low-resource language[13].

This classification stems from the nature of the language, in that there are two main forms of Arabic; Colloquial Arabic, and Modern Standard Arabic (MSA). The documentation of the two forms varies wildly, and many colloquial forms have little to no digital resources available. The main differences between the two forms lies in their usage and grammatical structure. Colloquial Arabic is highly regionally dependent, as it consists of all spoken dialects, and oftentimes the dialects are not mutually intelligible [10]. All dialects, however, share MSA as a basis for their linguistic features. These dialects all also omit key parts of the grammatical structure of MSA, namely the ending diacritical marks, also known as I'rab[15] in Arabic. I'rab defines MSA and its grammatical structure, and is one of the main difficulties for both native speakers and learners alike, functioning as a visual and auditory signal for case endings, ownership, morphology, and meaning.

Most of I'rab hinges on the short vowels of Arabic, referred to oftentimes as "harakat", which are the very diacritical marks that will be the focus of the paper. These are expected to be implicitly known when reading or speaking in MSA, and are almost always omitted in Arabic text, with the sole exception of children's educational text, and the Quran[12]. Since almost all written Arabic text is in MSA, this poses a large problem of interpretation of text, leading to numerous issues that include hindrance of progress in Arabic NLP.

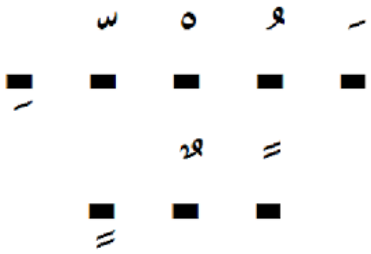


Figure 1: The main diacritical marks of Arabic

1.1. Demonstration

In Arabic NLP fields, most of the focus is dedicated to MSA, as colloquial Arabic has much less documentation in the written form associated with it. However, with the lack of diacritical marks,

the meaning of the words becomes muddled, leading to large assumptions and inferences being posed when processing any unmarked Arabic text. One can see this with the word ذهب[7], which in this current form has no indicated meaning. Adding some diacritical marks, we get the word ذَهَبَ (ða-ha-ba in IPA) , which can mean "he went" when used as a verb. Changing the last diacritical mark, we can obtain ذَهَبٌ (ða-ha-bun), which means gold in the indefinite, nominal case. This demonstration shows the exact hurdle that is created by the omission of the diacritical marks, and for larger texts a much larger problem appears in the scope of comprehension. This problem begins to encroach on the translation discourse that permeates Arabic literature, politics, and documentation[14].

2. Background and Related Works

Studies on machine translation discourse must first evolve through the stage of tackling which form of Arabic the model will rely on; MSA, or a form of colloquial Arabic. Some previous studies which have begun to tackle this are Faheem et al. (2024)[8], which attempts to improve the translation of the Egyptian dialect to MSA. The paper proposes a change in collection of corpus data, in order to overcome the barrier of Arabic's status as a low-resource language, and attempting to test them on three different translation models. Similarly, in Alnassan (2023) [5], the paper tackles the issue of "standardization" of dialects into MSA, in order for the translation from Arabic to English to be much more formal and efficient, rather than tackling different models, such as the methodology proposed in Faheem et al. Both of the drawbacks of the paper rely on the focus of one specific dialect (Egyptian and Syrian in these cases), which Kwaik et al. (2018) [14] foresees through the calculations done on the distances and differences between each dialects, and how this extends to NLP and ML usage in regards to Arabic.

The overall consensus (and general upheld practice) is conducting translation from MSA to English rather than from an Arabic dialect directly into English, which runs into the issue of how to apply Arabic grammatical theory into the models. In Halabi et al. (2020) [9], the paper proposes

the usage of treebanks alongside POS-tags as as a means to solve the low-resource language issue. The proposed treebank, I3rab, attributes its improvements to the close readings of classical Arabic grammatical theory, as opposed to the other treebanks who have relied on a modified form of grammar syntax to simplify translation into English. In Abdelkader et al. (2024) [1], the evolution of treebanks is also discussed as a means of choosing a proper corpus for a theoretical model. The paper aims to study the Jordan Comprehensive Contemporary Corpus, and attempts to annotate them using a modified POS-tagging system. This paper also encroaches on the concept of diacritical marks as a key element of MSA, and the POS system developed in this paper aims to become a step towards a model that may automatically add these diacritical marks, but does not do so within the scope of said paper. In Manaa et al. (2022) [16], an attempt at adding diacritical marks via a simplified POS-tagging system is discussed, citing an 88.33 percent accuracy. However, the system itself was tested on a small sample size of 300 sentences exclusively from Saudi school Arabic grammar books. The scope of this paper makes sense in regards to students within the Arab countries, however in order for the model to be applicable in a broader aspect, the sample testing size should not be limited in genre nor size.

The final step would be the application of these models onto Arabic-English translation. A study of the efficiency of various approaches and models is outlined in Babaali et al. (2023) [6], which reviews the various Arabic-English ML models, categorizes them based on if the models are lingustic based, corpus based, or both. The overview of efficiency is a key component to consider for the approach of this paper. In Alabdullah et al. (2025) [4], the paper focuses on the step before, in regards to the discussion of "standardization" of Arabic before it is translated into English. One of the key factors noted that allows for improvement of this step is the "fine-tuning" of LLMs, one method being the modification of POS-taggers (whether lexical simplification or increase in complexity). In Abu-shquier et al. (2017) [17], the paper provides an alternate means to the simplification of the model, which involves tackling the issue of word order rather than POS-tagging. The results of the model scoring above the other test models is attributed to such

an approach, however the grammar discussed within the paper is oversimplified, which raises the question of the scope of such a model on more complex MSA forms. In Inoue et al. (2021) [11], these factors are all discussed in the making of the CAMeLBERT systems, which is shown within the paper to not only work on MSA to English translation, but also direct dialect translation.

Overall, the research shown above is either lacking in resources, has used a different implementation than the one being considered, or have not delved fully into the specific goal of adding diacritical marks to Arabic text. In particular, research into models that predict diacritical marks is very sparse and lacking any complexity in the manner of training and prediction beyond a general pattern-learning approach.

3. Approach

3.1. Overview

One key assumption the overall model hinges on is the accuracy of the first layer in predicting Parts-Of-Speech (POS), as the model will later recompile to label the second dataset with POS-tagging. This tactic is implemented due to the sparsity of Arabic datasets and limited time, as there exists no dataset that has both diacritical marks included and is POS-tagged, and limitations on recreating a new dataset using a POS-tagger stems from lack of time to check the accuracy of the outputted results.

The second assumption relies on the nature of Arabic grammar and I'rab, as ending diacritical marks are an indicator of grammatical case[15]. Parts of speech is one of the major contextual pieces that defines these grammatical cases, and as such we predict that the added context will increase accuracy when balanced properly in the final joint Viterbi layer.

3.2. Parts-Of-Speech Hidden Markov Model

The first layer of the model is built on training on a dataset that has its Parts-Of-Speech (POS) labeled and annotated. The dataset used is the Kalimat Multicorpus Dataset[2], with usage of the POS-tagged corpus. This dataset has around 21,000 news articles that have been annotated, with a wide range of genres encompassed in the data. The main layer's focus is to properly build the dataset via an NLP pipeline to clean up the data, and then to train a simple HMM to learn the initial probability, transition probabilities, and emission probabilities of the (word, mark) pairings. The limitation of existing datasets that are both POS-tagged and diacritically-marked lead us to rely heavily on the accuracies of this layer, as later we will retrain the POS-HMM to the second dataset. We then evaluate this layer in order to gauge any possible limitations of the model that may restrict overall scores in the joint layer.

The pseudocode for the Hidden Markov Model is shown in **Fig. 2**:

```
HMM(sentences, k):

   Initialize counts: init, transition, emission, tag_counts
   Initialize sets: tags, vocab

   FOR each sentence:
       count initial tag
       FOR each (word, tag):
           update tag_counts, emission_counts
           add to tags and vocab
           update transition_counts (from previous tag)

   Compute init_prob with smoothing

   FOR each tag:
       Compute transition probabilities with smoothing
       Compute emission probabilities over vocab with smoothing
       Set emission(tag, "<UNK>")

   RETURN init_prob, trans_prob, emit_prob, tags, vocab
```

Figure 2: The basic HMM function that underlays Layer 1 and Layer 2

With this layer, the function would train on (word, POS) pairs. A quick simple Viterbi decode

```
VITERBI(sequence, model):

    GET INITIAL, TRANSITION, AND EMISSION PROBABILITIES
    CREATE V TABLE AND BACKPOINTER TABLE

    INITIALIZE FIRST WORD:
        FOR EACH STATE:
            SCORE = LOG(INITIAL PROBABILITY) + LOG(EMISSION PROBABILITY)
            STORE SCORE IN V[0]
            STORE NONE IN BACKPOINTER[0]

    FOR EACH REMAINING WORD:
        FOR EACH CURRENT STATE:
            CHECK ALL PREVIOUS STATES
            CHOOSE THE PREVIOUS STATE WITH THE HIGHEST SCORE:
                PREVIOUS SCORE + TRANSITION SCORE + EMISSION SCORE
            STORE THE BEST SCORE IN V
            STORE THE BEST PREVIOUS STATE IN BACKPOINTER

    CHOOSE THE BEST FINAL STATE FROM THE LAST V TABLE

    FOLLOW BACKPOINTERS BACKWARD TO BUILD THE BEST PATH

    RETURN BEST PATH
```

Figure 3: Pseudocode for the basic Viterbi decode function

(shown in **Fig. 3**) run on the training and testing set shows an overall accuracy above 90%.

3.3. Basic Word-Mark Hidden Markov Model

Due to a lack of datasets that are annotated with both POS-tagging and diacritical marks, we use a second dataset that has been fully marked, namely the Arabic Text Diacritization Dataset[7], extracted from the Tashkeela Corpus [20], which consists of over 2.3 million fully diacriticized Arabic words. We run this dataset through an NLP pipeline, which for this layer consists of stripping the word of all diacritical marks present, keeping the last diacritical mark (or the absence of one), and constructing the data in a (word, mark) tuple pairing. We define an ending diacritical mark not as the last present mark in a word, but the mark that is on the last letter of the word.

Similar to the first layer, the second layer construct a simple Mark HMM, which measures the basic probabilities and counts of the (word, mark) pairs. Passing in the second dataset, we obtain the basic word-mark HMM, which will construct the baseline of results to compare to for the joint layer. The HMM will have 9 possible "tags", eight of them shown in Fig. 1, with the addition of the absence of any ending diacritical mark as a possible tag.

3.4. Joint Viterbi Layer

The final layer takes into account the results of the two Hidden Markov Model layers. We annotate the second dataset with the simple Viterbi decode to obtain a complete dataset of (word, POS, mark) tuples, and retrain the first layer to obtain the (word, POS) probabilities of the second dataset. A key assumption held here is that the accuracy of the first layer is high enough that it will not heavily impact the joint layer accuracy.

Using the assumption that added context would increase accuracy of predictions of ending diacritical marks, we use the two layer HMM to calculate base probabilities, with a final modified Viterbi algorithm that calculates the final score using a weighted function that takes into account $P(\text{word}|\text{mark}), P(\text{mark}|\text{POS}), P(\text{mark}_\text{t}|\text{mark}_{\text{t}-1})$, and $P(\text{mark}|\text{word}, \text{POS})$. The overall formula is as follows:

$$\text{score}_{\text{overall}}(\text{mark}) = \text{dp}[t-1][\text{mark}] + \log(P(\text{mark}|\text{word}, \text{POS})) + \log(P(\text{mark}_\text{t}|\text{mark}_{\text{t}-1}))$$

where $\text{dp}[t-1][\text{mark}]$ is the dynamic programming table storing previous scores of the mark, $(P(\text{mark}|\text{word}, \text{POS}))$ is the combined emission probability, and $(P(\text{mark}_\text{t}|\text{mark}_{\text{t}-1}))$ is the (mark, mark) transition probability.

We estimate $P(\text{mark}_\text{t}|\text{word}_\text{t}, \text{POS}_\text{t})$, the combined emission probability, with the weighted score function as shown below:

$$\text{score}_{\text{emit}}(\text{mark}) = \alpha(\log(P(\text{word}|\text{mark}))) + (1-\alpha)(\log(P(\text{mark}|\text{POS})))$$

where alpha is a hyperparameter used to control the influence of the two emission probabilities of the two Hidden Markov Models.

The joint Viterbi function is largely the same as the basic Viterbi decode, except with the addition of the aforementioned weight function, as seen in Fig. 4

```
JOINT_VITERBI(words, POS, marks, POS_model, mark_model, alpha):

    INITIALIZE dp and backpointer tables

    # Weight Function
    DEFINE SCORE(word, pos, mark):
        RETURN α·log P(word|mark) + (1-α)·log P(mark|pos)

    INITIALIZE FIRST WORD:
      FOR each mark m:
        dp[0][m] = log P(start→m) + SCORE(word₀, pos₀, m)

    FOR EACH REMAINING WORD:
      FOR t = 1 to n-1:
        FOR each mark m:
            dp[t][m] = max over m' [
                dp[t-1][m'] + log P(m'→m) + SCORE(word_t, pos_t, m)
            ]
            back[t][m] = argmax m'

    CHOOSE THE BEST FINAL STATE FROM THE LAST V TABLE
    FOLLOW BACKPOINTERS BACKWARD TO BUILD THE BEST PATH
    RETURN BEST PATH
```

Figure 4: Pseudo-code for the Joint Viterbi function

4. Implementation

4.1. Overview

The model itself is implemented using Python [3], and leverages a combination of standard libraries for data processing, modeling, and evaluation. The NumPy library is used for efficient numerical operations, while built-in modules such as math support logarithmic computations for stable probability calculations in the Viterbi algorithm. Data handling and preprocessing are facilitated using collections (e.g., Counter and defaultdict) for frequency counting and structured storage of probabilities. Model evaluation is performed using scikit-learn, specifically for computing F-1 scores, generating classification reports, and constructing confusion matrices. Visualization of results, including confusion matrices and model diagrams, is handled using matplotlib, with

NetworkX used to represent and visualize the structure of the Hidden Markov Model. Progress tracking during data processing is supported by tqdm, while pickle is used for saving and loading intermediate model artifacts. Finally, pathlib is utilized for robust file system navigation and dataset management. Together, these libraries provide a lightweight yet flexible framework for implementing, evaluating, and visualizing the proposed HMM-based models.

Evaluation is done via the use of F-1 scores, confusion matrices, and accuracy scores. F-1 and the confusion matrices are implemented via sklearn, while accuracy is generated from raw comparisons between truth labels and predicted labels. The second layer results and the joint layer results will be compared to evaluate the overall performance of the model, with the second layer serving as a benchmark for scores.

4.2. Dataset Usage

For the first dataset, we implement the Kalimat Multicorpus Dataset, specifically using the corpus post-tag dataset. The dataset is processed using a custom file-reading function that iterates through all files in a specified directory and constructs sentence-level data from line-based annotations. Each line in a file is expected to follow a word/tag format, which is split to extract the word and its corresponding label. These pairs are appended to a sentence list, which are segmented based on punctuation markers (specifically when a period tagged as PUNC is encountered). Empty lines are ignored to ensure clean parsing. The result is a structured dataset represented as a list of sentences, where each sentence is a list of (word, tag) tuples. Finally, the dataset can be split into training and validation sets using sklearn.train-test-split for downstream modeling and evaluation, with a standard 80/20 split utilized for this specific implementation.

For the second dataset, we implmenet the Arabic Text Diacriticization dataset, which is fully annotated with all proper diacritical marks required. The fully-marked raw Arabic text is processed using a custom preprocessing pipeline designed to extract word–diacritic pairs suitable for sequence modeling. A predefined set of Arabic diacritics is used to distinguish between base letters and

diacritical marks. For each word, the function separates the base characters from their associated diacritics and retains only the final diacritic attached to the last letter, producing a normalized (word, mark) representation. The text is first split into sentences using punctuation-based rules, and each sentence is tokenized using regular expressions to isolate valid Arabic words. The processing pipeline then applies the diacritic extraction function to each token, resulting in a structured corpus represented as a list of sentences, where each sentence consists of (word, mark) pairs. This format enables consistent training and evaluation of the word–mark HMM while focusing specifically on predicting final diacritics. Careful handling of the raw Arabic text requires the usage of regex and hard-coding utf-8 in order to properly read in the dataset. The corpus is already presplit into test, train, and validation, so no extra work was required beyond the cleanup.

4.3. Word-POS HMM Layer

The first layer is implemented as a standard Hidden Markov Model trained on sequences of (word, POS) pairs. During training, the model computes initial, transition, and emission probabilities from frequency counts extracted from the training corpus. Initial probabilities are estimated from sentence-start tags, transition probabilities from consecutive tag pairs, and emission probabilities from (tag, word) co-occurrences. To ensure robustness, add-k (Laplace) smoothing is applied to all probability estimates (with k set to 0.001 for this implementation), including a dedicated <UNK> token to handle unseen words at inference time. The vocabulary and tag set are constructed dynamically from the training data.

For decoding, a standard Viterbi algorithm in log-space is used to determine the most likely sequence of POS tags for a given sentence. The implementation maintains dynamic programming tables to store the best log-probability scores for each state at each time step, along with backpointers for path reconstruction. At each step, the algorithm combines the previous state score with the transition and emission probabilities, selecting the maximum-scoring path. Logarithmic probabilities are used to prevent numerical underflow, and small constants are applied to avoid undefined values.

The final POS sequence is recovered through backtracking from the highest-scoring final state. This layer provides the POS predictions that are later used as contextual input in the joint model.

4.4. Word-Mark HMM Layer

The setup of the second layer reuses the exact same functions as those detailed in the first layer setup, except now conducted on the second dataset. Once both layers are complete, we move onto the final joint layer, which constructs the final complete dataset that will be used to confirm the assumption that added context increases performance of the model.

4.5. Joint Viterbi Layer

The standard Viterbi decode for POS-tagging is done on the second dataset, which we then retrain the first layer to get the probabilities/counts of POS-tagging on the second dataset. In this step, an important constraint arises, which is the use of self-training, which can hinder results (discussed in future work section). Once this is done, a complete dataset has been created for the joint layer, which is in the form of (word, POS, mark) tuples.

We now reach the key part of this model, which is the joint Viterbi decoder function. It predicts the most likely sequence of diacritic marks by combining lexical and POS-based information. For each word position, the model computes a weighted joint log score using P(word | mark) from the word–mark HMM and P(mark POS) from the POS-conditioned mark model. The weighting parameter α controls the relative contribution of these two signals (as seen in the weight function in the Approach section). The decoder then applies the standard Viterbi dynamic programming procedure: it initializes scores using mark start probabilities, recursively combines previous mark scores with transition probabilities and joint emission scores, and stores backpointers to recover the best path. Finally, the highest-scoring final mark is selected, and the optimal mark sequence is reconstructed through backtracking. This allows the model to incorporate both word-level evidence and POS context when predicting final diacritics.

Finally, we evaluate the results of each dataset relative to the set alpha hyperparameter. The

accuracies for all alpha results for the test, train, and validation sets are stored, and accuracy scores are then calculated. We compare the results to layer 2 (equivalent to $\alpha = 1$), and compute F-1 scores and confusion matrices to determine the effect of the weight function.

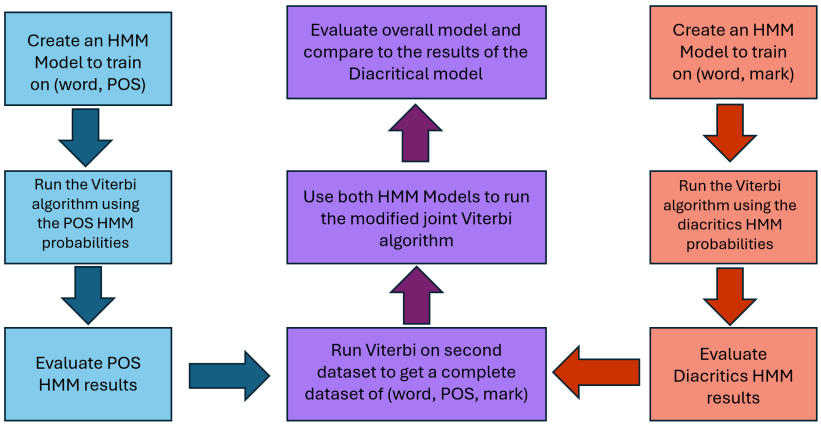


Figure 5: Visualization of the 2-layer HMM and the final Joint Layer

5. Evaluation

5.1. First Layer

Table 1: Accuracy Scores for Layer 1

	Overall Accuracy	Known Accuracy	Unknown Accuracy
Training	93.7%	93.7%	N/A
Testing	92%	92.9%	11.87%

We run the first layer, which is the word-POS Hidden Markov Model layer built on the training of the Kalimat Multicorpus Dataset, and obtained overall accuracies of over 90%, as seen in **Table 1**. We note the low accuracy scores for the unknown accuracy, a limitation that is noted down for later layers of the model. However, the number of unknown words appears to be very little, as it does not impact the known accuracy scores when calculating the overall accuracy.

The Markov Chain for the hidden states for the first layer is shown in **Fig 6**, which has been culled to show the top 10 most frequent POS-tags, with a threshold of 0.1 for the transition probabilities that are to be displayed, as the set of all possible POS-tags is the 31 tags of the Arabic Penn Treebank. We note that the text is heavily weighted towards nouns, which indicates a key feature of MSA in its grammatical structure and morphological syntax. The graph is highly connected, indicating diverse tag transitions. Certain high-probability transitions such as DT $\rightarrow$ NN and NN $\rightarrow$ VB reflect very common syntactic structures of MSA deriving from nominal and verbal sentences.

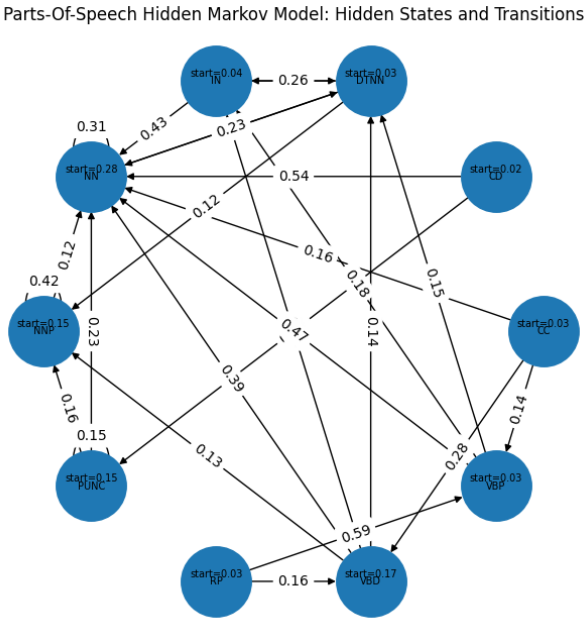


Figure 6: Visualization of the Markov Chain of the Top 10 Hidden States of the First Layer

Table 2: F-1 Scores for Layer 1

	Micro F-1 Score	**Macro F-1 Score**	**Weighted F-1 Score**
Training	93.68%	92.07%	93.66%
Testing	91.97%	83.43%	92%

Using F-1 Scores as another means of evaluation, we can see from **Table 2** that micro and weighted F-1 scores are high for both the training and testing dataset. However, the macro F-1

score shows a substantial drop, particularly with the 83% score for the testing dataset, indicating degraded performance on less frequently seen classes. We can therefore surmise that the model is heavily biased towards the more common tags, and struggles with rare or unseen instances. When comparing to the macro F-1 score of the training dataset, we can conclude that the drop for the testing score arises from a generalization issue on rarer tags.

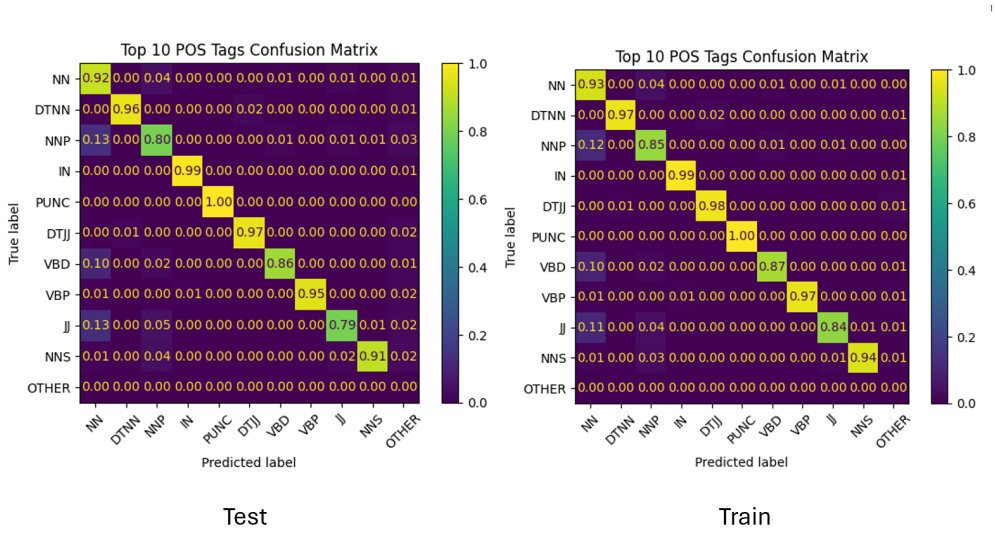


Figure 7: Layer 1 Confusion Matrix

Using confusion matrices as a more visual form of evaluation, we compare the two results of the test and train matrices in **Fig. 7**. Similar to the Markov chain, the matrices have been reduced to the top 10 most common POS tags, as a means of readability for this paper. Comparing the diagonals of the train vs test matrix, we note that the values are extremely similar, meaning that the model is able to generalize well between the train and test dataset. One key value to analyze is the model's confusion between NNP and NN, indicating confusion between proper nouns and common nouns. This stems from the manner in which a noun in Arabic is classified as definite or indefinite, as a prefix is usually attached to indicate definiteness, but oftentimes is omitted as the noun derives its definiteness from a larger morphological syntax present. Overall, we can conclude that these matrices show strong diagonal dominance, indicating high overall tagging accuracy and good generalization, and the similarity between the two matrices suggests minimal overfitting. Certain errors made by the model arise from linguistic ambiguity in Arabic. The consistency of

misclassification patterns across training and testing suggests that errors are driven by intrinsic ambiguity in the data rather than model instability. Overall, the first layer performed well enough to hold up one of the two key assumptions held for the overall model.

5.2. Second Layer

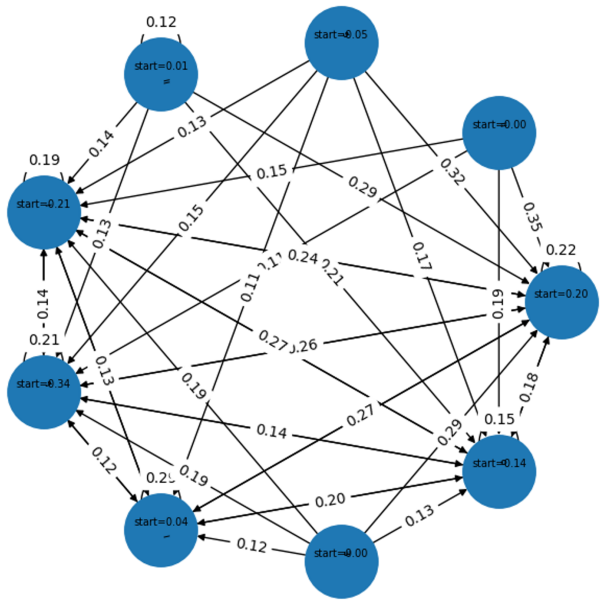


Figure 8: Simple Word-Mark Markov Chain

We run the second layer to obtain a simple word-mark Hidden Markov Model. As shown in **Fig. 8**, we are able to see the transitions between all 9 possible ending diacritical marks. This graph is much more connected than layer 1, which reflects the nature of case endings and I'rab. We note that the states with the highest starting probabilities are the markings that reflect the definitive, nominal case, and that certain markings have no incoming arrows. These transitions reflect key parts of the grammatical structure of Arabic, and for the sake of simplicity will not be delved into, however the manner in which certain marking transition pairs are asymmetric highlight once again more of the morphological and grammatical syntax at play here.

The word–mark HMM demonstrates stable and consistent performance across splits as seen in **Table 3**, with overall accuracy remaining close between training (81.29%), validation (79.02%), and testing (78.8%), indicating minimal overfitting and good generalization. Performance on known

Table 3: Accuracy Scores for the Second Layer

	Overall Accuracy	Known Accuracy	Unknown Accuracy
Training	81.29%	81.29%	N/A
Testing	78.8%	80.22%	22.58%
Validation	79.02%	80.12%	27.78%

words is strong and nearly unchanged across datasets (80–81%), showing that the model effectively captures frequent word–mark patterns. However, there is a pronounced drop in unknown word accuracy, particularly on the test set (22.58%) and, to a lesser extent, the validation set (27.78%). This gap highlights a key limitation of the model: its heavy reliance on observed word–mark emissions, which do not generalize well to unseen vocabulary. Consequently, while the model performs reliably on familiar inputs, its ability to handle rare or unseen words remains weak, suggesting that improvements such as enhanced smoothing, better UNK handling, or incorporation of additional contextual information may be necessary to improve robustness. Thus, we use these results as the baseline to compare to for the joint layer, as the underlying assumption for the project is the use of POS-tagging as partial additional grammatical context.

Table 4: F-1 Scores for the Second Layer

	Micro F-1 Score	Macro F-1 Score	Weighted F-1 Score
Training	81.65%	64.25%	82.42%
Testing	79.23%	62.5%	80.06%
Validation	79.02%	61.77%	80.16%

F-1 scores further highlight the imbalance in model performance across classes. While the micro and weighted F-1 scores remain relatively high (79–81%), indicating strong performance on frequent marks, the macro F-1 score is significantly lower (62–64%). This substantial gap suggests that the model struggles with less frequent classes, as macro F-1 assigns equal importance to all labels regardless of frequency. The consistency of this pattern across training, testing, and validation sets indicates that the issue is not due to overfitting but rather reflects an inherent limitation of the word–mark HMM in handling rare or unseen marks. This observation is consistent with the low unknown-word accuracy.

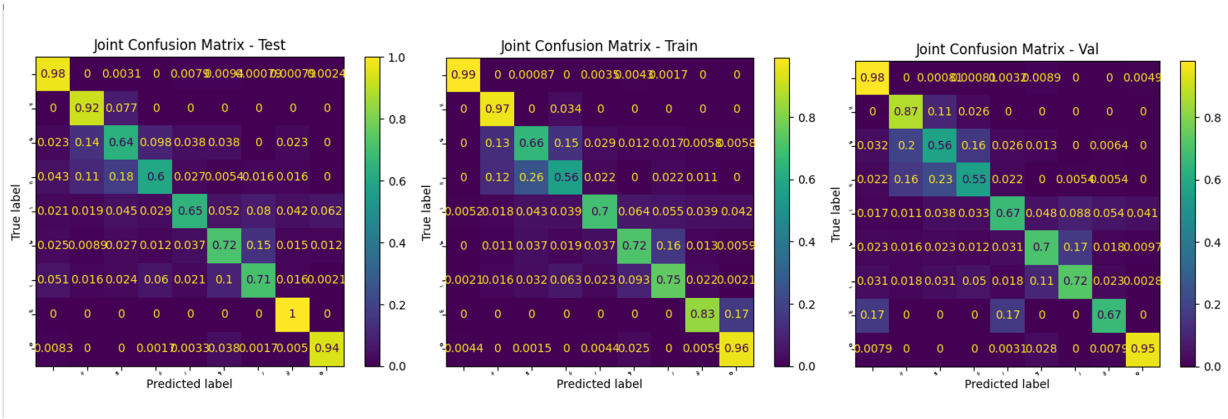


Figure 9: Confusion Matrices for Second Layer

As showin in **Fig. 9**, the confusion matrices for training, testing, and validation (joint $\alpha = 1$, equivalent to the second layer) exhibit strong diagonal dominance, indicating that the combined model achieves high overall accuracy across datasets. The similarity between the three matrices suggests consistent generalization, with no significant signs of overfitting. However, compared to the POS-only model, the diagonal entries for several classes are lower and more diffuse, reflecting increased confusion introduced by the joint decoding process. In particular, mid-frequency classes show noticeable off-diagonal mass, indicating systematic misclassification between related categories, while dominant classes remain highly accurate and continue to act as attractors for errors. This pattern highlights a trade-off introduced by the joint model: while it leverages additional POS information, it also propagates uncertainty across layers, leading to more distributed errors. Nonetheless, the consistency of these confusion patterns across splits suggests that the errors are driven by inherent ambiguity in the data rather than instability in the model. Further testing shows that the lower-performing tags are misclassified by their opposite definiteness mark counterpart, further indicating the model's weakness towards proper identification of definiteness.

5.3. Joint Viterbi Layer

In **Table 5**, the results clearly show the impact of the weighting parameter α on the joint model's performance. Increasing α from 0.2 to 0.6 leads to consistent improvements across all metrics and datasets. We chose these particular values in order to sample the change in accuracy given low-high alpha values. In particular, overall accuracy rises from 77.1% to 81.44% on the test set

Table 5: Sample Accuracies of the Joint Layer

$\alpha = 0.2$			
	Overall	**Known**	**Unknown**
Training	78.72%	78.72%	N/A
Testing	77.1%	78.27%	31.45%
Validation	78.3%	79.25%	34.26%

$\alpha = 0.6$			
	Overall	**Known**	**Unknown**
Training	83.3%	83.3%	N/A
Testing	81.44%	82.7%	32.25%
Validation	81.42%	82.4%	35.19%

and from 78.3% to 81.42% on the validation set, while known-word accuracy shows similar gains (e.g., 78.27% to 82.7% on test). Notably, unknown accuracy also improves slightly (31.45% $\rightarrow$ 32.25% on test, and 34.26% $\rightarrow$ 35.19% on validation), indicating that emphasizing the P(wordmark) component (higher α) benefits both frequent and unseen cases. This suggests that the word–mark signal is more informative than the POS-based component in the joint model, and that giving it greater weight leads to better overall performance. The consistency of these improvements across training, testing, and validation sets further indicates that the effect is robust rather than due to overfitting.

Comparing the joint model results to the standalone word–mark HMM reveals that incorporating POS-based context provides a clear overall benefit, but with nuanced effects across different subsets of the data. Relative to the baseline (78.8% test accuracy and 22–28% unknown accuracy), the joint model improves overall accuracy (up to 81.44%) and known-word accuracy (up to 82.7%), indicating that the added POS information helps disambiguate common and moderately frequent cases. This suggests that the joint formulation is successfully leveraging contextual signals to refine predictions where the word–mark model already has strong evidence.

However, the gains in unknown accuracy are more modest (22–28% $\rightarrow$ 31–35%), and remain substantially lower than known accuracy. This indicates that while the added POS context does improve generalization to unseen words, it does not fully resolve the model's dependence on observed word–mark emissions. In other words, POS information provides useful but incomplete context for handling rare or unseen cases. The boost gained is still significant enough for the purposes of this paper, however other methods of additional context may increase unknown accuracy even further.

Table 6: Sample Accuracies of the F-1 Scores

$\alpha = 0.2$				$\alpha = 0.6$			
	Micro	**Macro**	**Weighted**		**Micro**	**Macro**	**Weighted**
Training	78.72%	52.85%	77.67%	**Training**	83.3%	72.75%	83.5
Testing	77.1%	49.32%	75.77%	**Testing**	81.44%	68.28%	81.57%
Validation	78.3%	52.96%	77.27%	**Validation**	81.42%	67.87%	81.65%

The comparison between α =0.2 and α =0.6 further reinforces this interpretation: higher α (placing more weight on P(wordmark)) yields better performance across all metrics, suggesting that the word–mark signal remains the dominant contributor. Thus, the added POS context acts as a complementary signal that enhances performance, particularly for known words, but does not replace the primary role of lexical information.

The F-1 scores further highlight the impact of incorporating POS-based context in the joint model, as seen in **Table 6**. When α=0.2, performance degrades across all metrics, with a particularly sharp drop in macro F-1, indicating that over-reliance on POS information negatively affects both frequent and rare classes. In contrast, setting α=0.6 yields consistent improvements over the baseline word–mark HMM, increasing micro, macro, and weighted F-1 scores across all datasets. Notably, the macro F-1 score shows a substantial gain, suggesting that the joint model improves performance on less frequent marks. This demonstrates that POS information provides valuable contextual cues, but only when balanced appropriately with lexical information, reinforcing the importance of weighting in the joint formulation.

The confusion matrices for α=0.2 as shown in **Fig. 10** reveal a significant degradation in model performance, characterized by reduced diagonal dominance and increased dispersion of probability mass across multiple classes. Unlike the baseline and higher-α configurations, predictions are less concentrated, indicating greater uncertainty and weaker class separation. This is particularly evident in mid-frequency classes, where predictions are distributed across several competing labels rather than forming a clear peak. The consistency of this pattern across training, testing, and validation sets

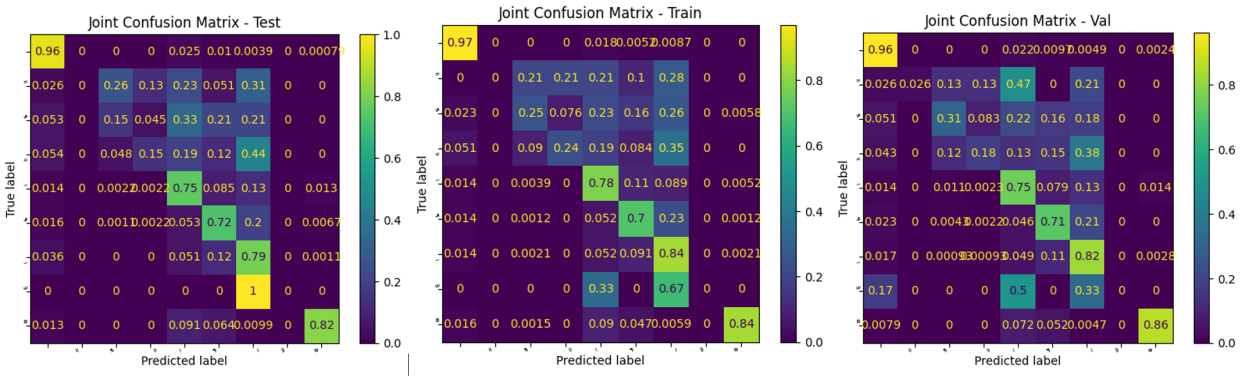


Figure 10: Confusion Matrices for the Joint Layer ($\alpha = 0.2$)

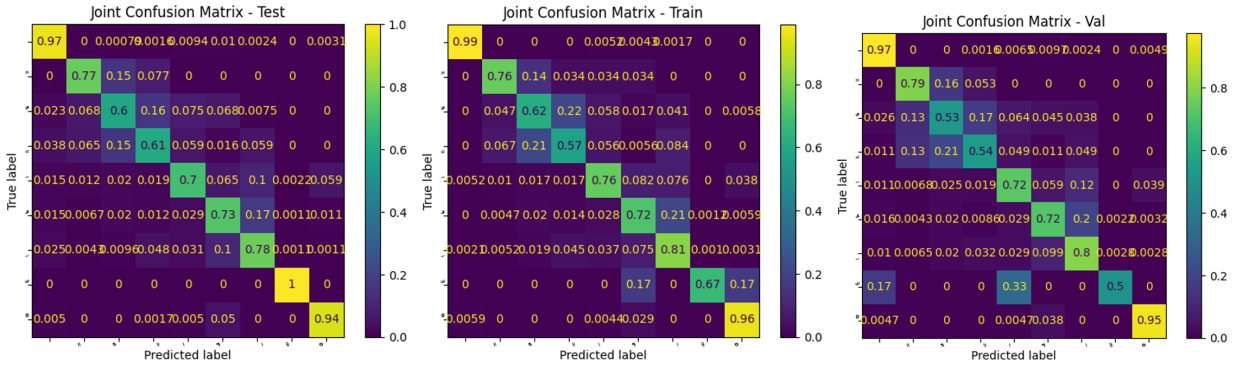


Figure 11: Confusion Matrices for the Joint Layer ($\alpha = 0.6$)

suggests that the issue arises from the model formulation rather than overfitting. Specifically, the heavy reliance on P(mark | POS) reduces the influence of lexical information, leading to increased ambiguity and degraded performance across both frequent and rare classes.

In comparision, in **Fig. 11** the confusion matrices for α=0.6 demonstrate a substantial improvement over the α=0.2 configuration, with a clear restoration of diagonal dominance across all datasets. Predictions are more concentrated, indicating stronger class separation and reduced uncertainty. Compared to the diffuse patterns observed at lower α, mid-frequency classes exhibit significantly higher diagonal values, reflecting improved discrimination and more consistent predictions. Off-diagonal entries are reduced and more structured, suggesting that remaining errors are systematic rather than due to ambiguity. The consistency of these patterns across training, testing, and validation sets confirms that the improvement is robust. These results indicate that emphasizing the lexical component while incorporating POS information as a secondary signal leads to a more effective and stable joint model.

5.4. Overall

The overall results demonstrate that incorporating POS-based context into the word–mark HMM can improve performance, but only when appropriately balanced. The baseline word–mark model achieves strong overall and weighted performance (79–81% micro/weighted F-1) but exhibits a notable gap in macro F-1 (62–64%), reflecting difficulty with rare marks. Introducing the joint model with α=0.2 degrades performance across all metrics, with micro and weighted F1 decreasing and macro F-1 dropping sharply to 49–53%. The corresponding confusion matrices show diffuse predictions and reduced class separation, indicating that over-reliance on P(mPOS) introduces ambiguity and weakens lexical discrimination. In contrast, the joint model with α=0.6 outperforms the baseline across all metrics, increasing micro and weighted F-1 to 81–83% and substantially improving macro F-1 to 68–73%. The confusion matrices at this setting exhibit stronger diagonal dominance and more concentrated predictions, confirming improved class separability. Notably, unknown-word accuracy also improves relative to the baseline, indicating better generalization. Together, these results show that POS information provides valuable contextual cues, but only when used as a complementary signal to the dominant word–mark probabilities; excessive reliance on POS degrades performance, while a balanced weighting yields consistent gains across both frequent and rare classes. Thus, the initial assumption is proven true, when carefully applied in the context of the model's framework.

6. Conclusion and Future Work

6.1. Conclusion

In this work, we introduced a two-layer Hidden Markov Model for predicting ending diacritical marks in Arabic text, combining a word–mark HMM with POS-based contextual information through a joint Viterbi decoding framework. The results demonstrate that incorporating grammatical context can improve performance, particularly when balanced appropriately with lexical information. While the baseline word–mark model provides strong performance on frequent patterns, it

struggles with rare and unseen cases. The joint model, when tuned with an optimal weighting parameter (e.g., $\alpha = 0.6$), achieves consistent improvements in overall accuracy, F-1 scores, and unknown-word performance, confirming that POS context serves as a valuable complementary signal. However, the findings also highlight that over-reliance on POS information can degrade performance, emphasizing the importance of careful model calibration. Overall, this study supports the hypothesis that integrating linguistic structure enhances diacritic prediction, while also pointing to the need for further improvements in handling data sparsity and unseen vocabulary in Arabic NLP tasks

6.2. Limitations and Future Work

As discussed in the paper, the main limitation lies in the self-training of the second dataset. No dataset exists (or was found) that is POS-tagged completely, has proper diacritcization, and is large enough to comfortably generalize the model outputs with. Time constraints also limit the possibility of creating a new model via a proper POS-tagger (such as the Stanford POStagger[18]), as one would have to independently verify the results, which lead to either having to choose between a smaller curated dataset or the larger self-trained dataset. The evaluation of the first layer showed that the POS Hidden Markov Model performed extremely well, however further testing can be done on other POS-tagged datasets to check the full performance of the first layer.

This limitation can be solved in future work, where enough time is given to properly curate and create a proper dataset to train the model on. Another aspect of the model that can be improved upon is the manner in which additional context is defined. One method that was proposed but ultimately scrapped was the usage of morphological analysis, however the complexity of the computation would drastically increase. The basis of the model is another aspect to consider for future work, such as using Recurrent Neural Networks (ex. Long-Short Term Memory Models (LSTM models)), or expanding this towards new developmental models in NLP. The application of this model as a intermediate step in Arabic-English translation models is also a possible application, given that the

underlying goal of this project is the addition of the ending diacritical marks as a means of adding meaning and grammatical context.

7. Acknowledgments

I would like to thank my COS Advisor Dr. Christiane Fellbaum for her advice and mentoring throughout this project, and for listening to my complaints about not having high enough accuracy scores. I would also like to thank my NES Advisor for this project, Dr. Mounia Mnouer, who advised with the Arabic linguistic and grammar portion of this project, and aided in possible implementation ideas to use for the additional context caveat.

8. Ethics

This paper represents my own work in accordance with university regulations.

- Ayaat Al-Yasseri

References

[1] Y. S. M. . S. M. S. Abdelkader, D. O., "Sources of syntactic ambiguity in contemporary arabic: A corpus study," *Jordanian Educational Journal, 9(3), 27–46.*, 2024.

[2] F. Al-Dawsari, "Kalimat multipurpose arabic corpus files," https://www.kaggle.com/datasets/ferasaldawsari/kalimat-multipurpose-arabic-corpus-files.

[3] A. Al-Yasseri, "Auto-i3rab," 2026. [Online]. Available: https://github.com/aa1461/AUTO_I3RAB

[4] A. Alabdullah, L. Han, and C. Lin, "Advancing dialectal arabic to modern standard arabic machine translation," 2025. [Online]. Available: https://arxiv.org/abs/2507.20301

[5] A. Alnassan, "Automatic standardization of arabic dialects for machine translation," 2023.

[6] B. BABAALI and M. SALEM, "Arabic machine translation: A panoramic survey," *Independent*, 2023.

[7] A. Fadel, I. Tuffaha, B. Al-Jawarneh, and M. Al-Ayyoub, "Arabic text diacritization," https://github.com/AliOsm/arabic-text-diacritization, 2019.

[8] W. K. B. H. e. a. Faheem, M.A., "Improving neural machine translation for low resource languages through non-parallel corpora: a case study of egyptian dialect to modern standard arabic translation." *Sci Rep 14, 2265*, 2024.

[9] D. Halabi, E. Fayyoumi, and A. Awajan, "I3rab: A new arabic dependency treebank based on arabic grammatical theory," 2020. [Online]. Available: https://arxiv.org/abs/2007.05772

[10] Industry Arabic, "The industry arabic guide to arabic dialects," https://industryarabic.com/arabic-dialects/.

[11] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, "The interplay of variant, size, and task type in Arabic pre-trained language models," 2021.

[12] Kalimah Center, "Arabic harakat, tashkeel, and diacritics: Everything you need to know," https://kalimah-center.com/arabic-harakat-tashkeel-diacritics/, 2025.

[13] R. Karam, "Is arabic really a well-resourced language? digital approaches to middle arabic and beyond," *Connections: A Journal for Historians and Area Specialists*, 2026. [Online]. Available: https://www.connections.clio-online.net/article/id/fda-161011

[14] S. C. S. D. Kathrein Abu Kwaik, Motaz Saad, "Arabic machine translation: A panoramic survey," *Arabic Machine Translation: A Panoramic Survey*, 2018.

[15] Learn Through Lyrics, "I'rab / syntactic analysis / ," https://learnthroughlyrics.com/irab-%D8%A5%D8%B9%D8%B1%D8%A7%D8%A8/.

[16] Z. Mannaa, A. Azmi, and H. Aboalsamh, "Computer-assisted i'raab of arabic sentences for teaching grammar to students," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, 08 2022.

[17] M. M. A. Shquier and K. M. Alhawiti, "Fully automated arabic to english machine translation system: transfer-based approach of ae-tbmt," *Int. J. Inf. Commun. Technol.*, 2017.

[18] Stanford Natural Language Processing Group, "Stanford part-of-speech tagger," https://techfinder.stanford.edu/technology/stanford-part-speech-tagger.

[19] United Nations, "World arabic language day," https://www.un.org/en/observances/arabiclanguageday.

[20] T. Zerrouki, "Tashkeela: Arabic diacritization corpus," https://www.kaggle.com/datasets/linuxscout/tashkeela.